IAM (Identity Access Management)

Notes: *Create a Jira ticket for this assignment, work on the tasks listed below, and submit the task completion document by EOD with clear steps and required evidence (outputs/screenshots)*

1. Create one IAM user and assign EC2 and S3 full access roles.
2. Create one group in IAM and assign read access for EC2.
3. Create a new user named "Devops" and add to the group created in task 2.
4. Write a bash script to create an IAM user with VPC full access.
5. Create an IAM policy to allow EC2 access for a specific user in specific regions only.
6. We have two accounts: Account A and Account B. Account A user should access an S3 bucket in Account B. (Collaborate with a team member and execute this. This is mostly asked in every interview.)

1) 1. Create one IAM user and assign EC2 and S3 full access roles.

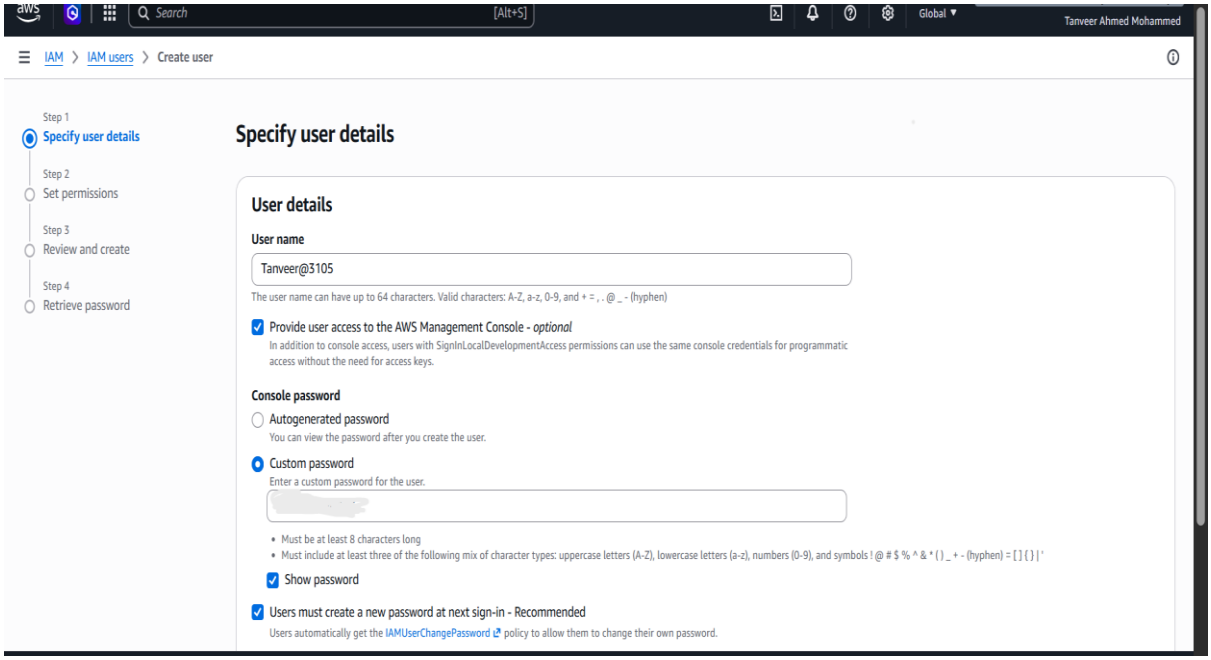
Create an IAM user and attach two AWS-managed policies.

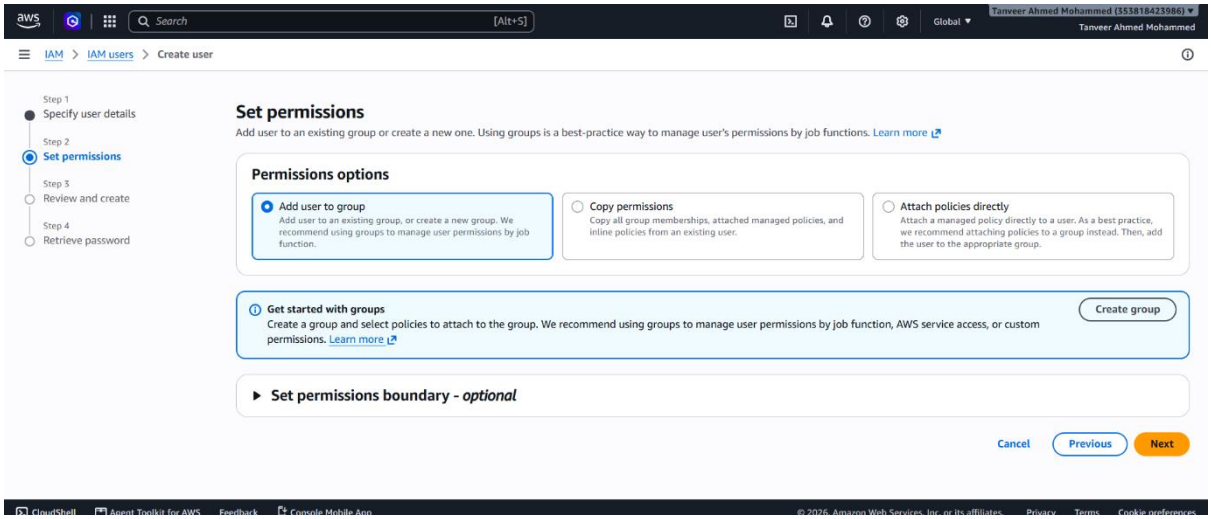
Step 1: Open IAM

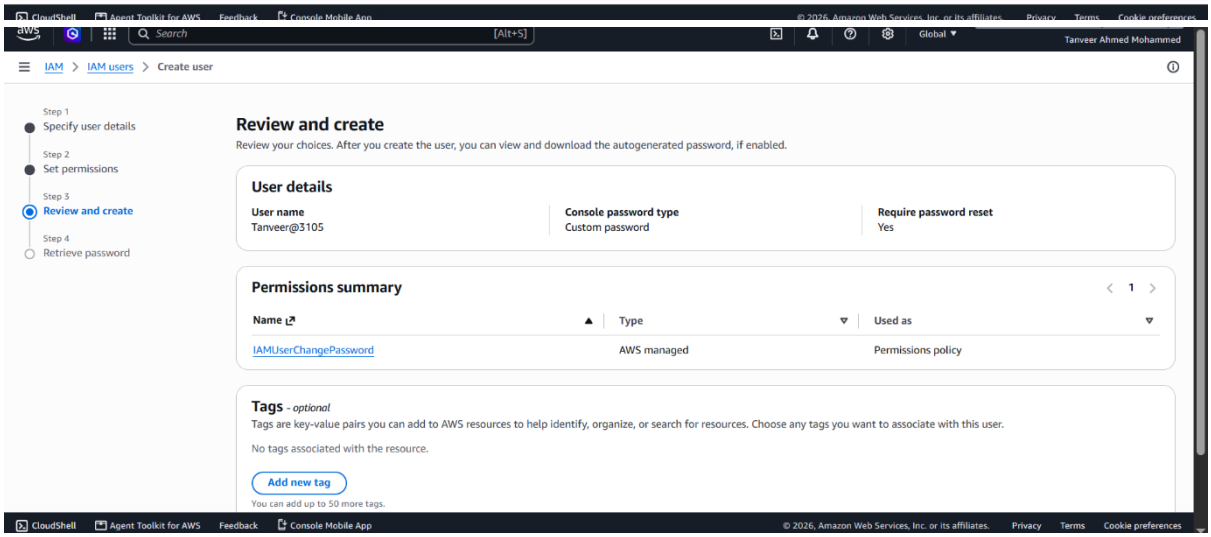
1. Login to AWS Management Console.
2. In the top search bar, type IAM.
3. Select Identity and Access Management.
4. From the left menu, click Users.

Step 2: Create the user

1. Click Create user.
2. Enter the username: Tanveer@3105
3.







 If you need console login, select the option to provide console access.

 For this lab, you can leave console access disabled if you are only testing through CLI.

 Click Next.

After selecting Attach policies directly, a policy list will appear.

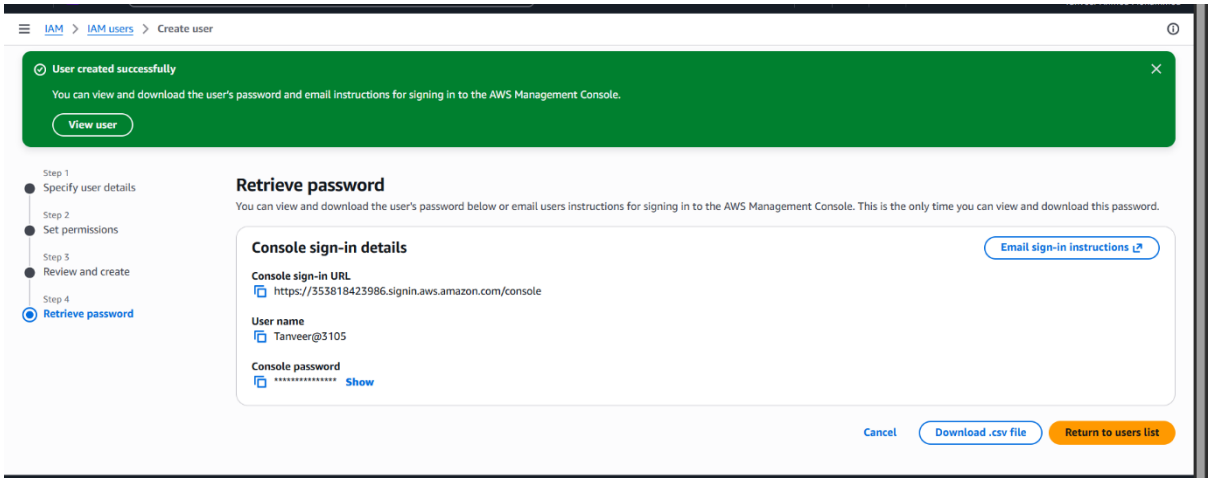
Search and select these two policies:

1. AmazonEC2FullAccess
2. AmazonS3FullAccess

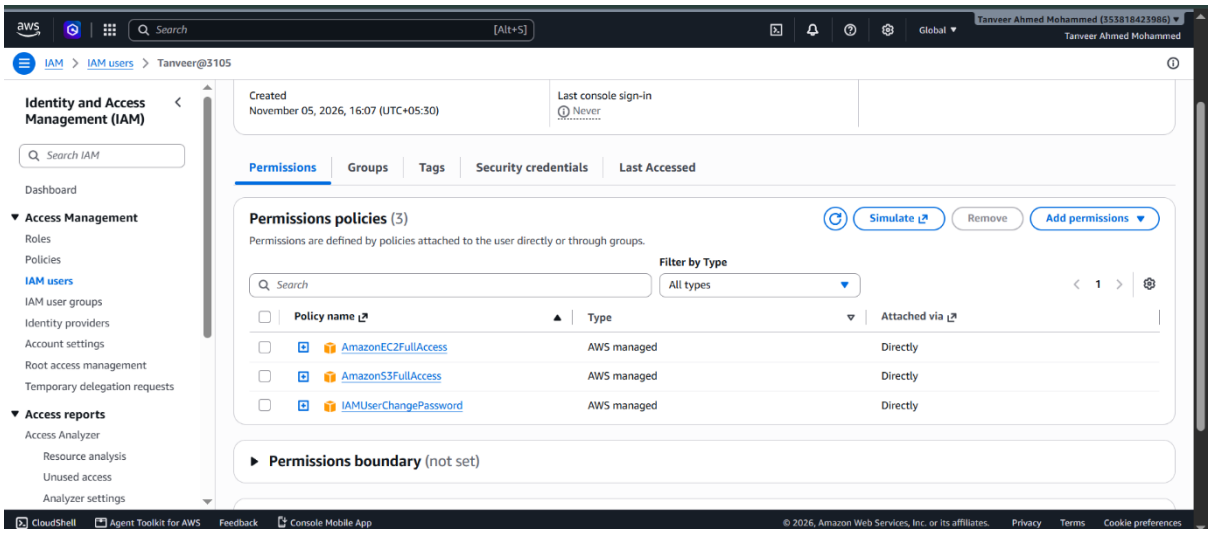
Make sure both checkboxes are selected.

Step 3: Click Next

1. Click Next (bottom right).
2. Review the username and permissions.
3. Click Create user.



4. **Step 4: Verify**
5. Go to:
6. `IAM → Users → Tanveer@3105 → Permissions`



2) 2. Create one group in IAM and assign read access for EC2.

Users added to this group will inherit EC2 read-only permissions.

Step 1: Open IAM User Groups

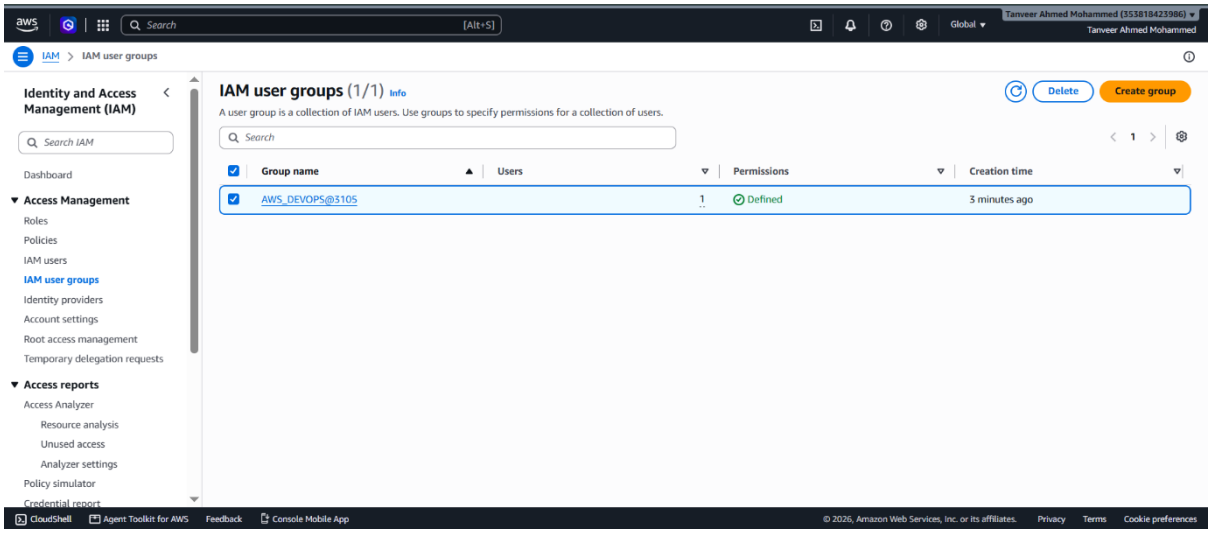
1. Go to AWS Console.
2. Search for IAM.
3. In the left-side menu, click User groups.
4. Click Create group.

Step 2: Enter the group name
AWS_DEVOPS@3105

Step 3: Attach EC2 Read-Only policy

Scroll down to Attach permissions policies.

AmazonEC2ReadOnlyAccess

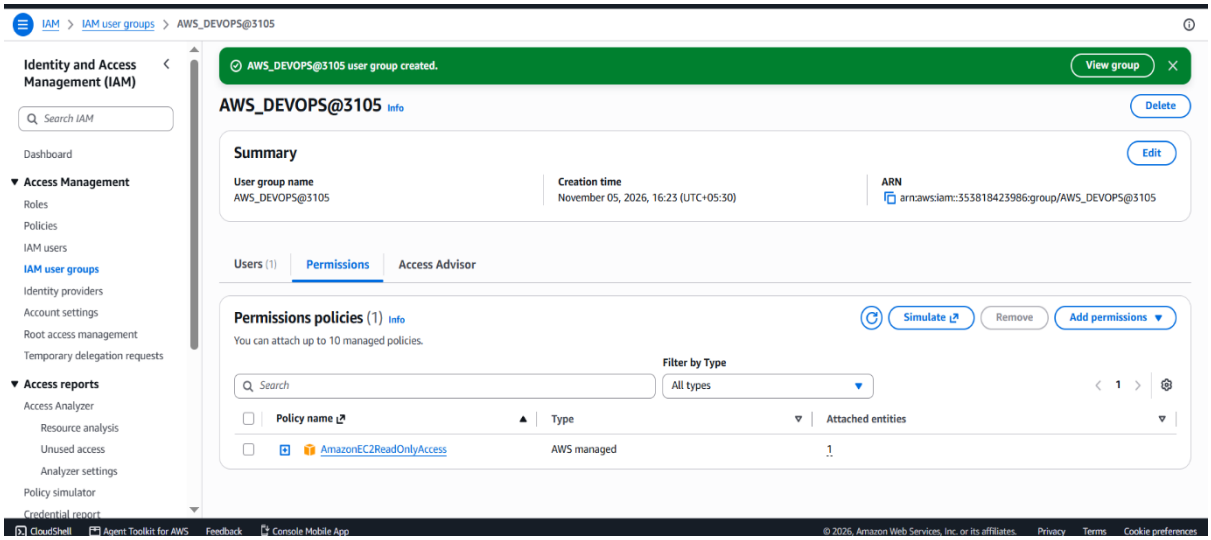


Step 4: Create the group

1. Check the group name.
2. Confirm the policy is selected.
3. Click Create user group.

4. Step 5: Verify the group

5. Navigate to:
6. `IAM → User groups → EC2-ReadOnly-Group`



3. Create a new user named "Devops" and add to the group created in task 2.

Objective: Create a new IAM user named Devops and add that user to the EC2-ReadOnly-Group created in Task 2.

Expected result

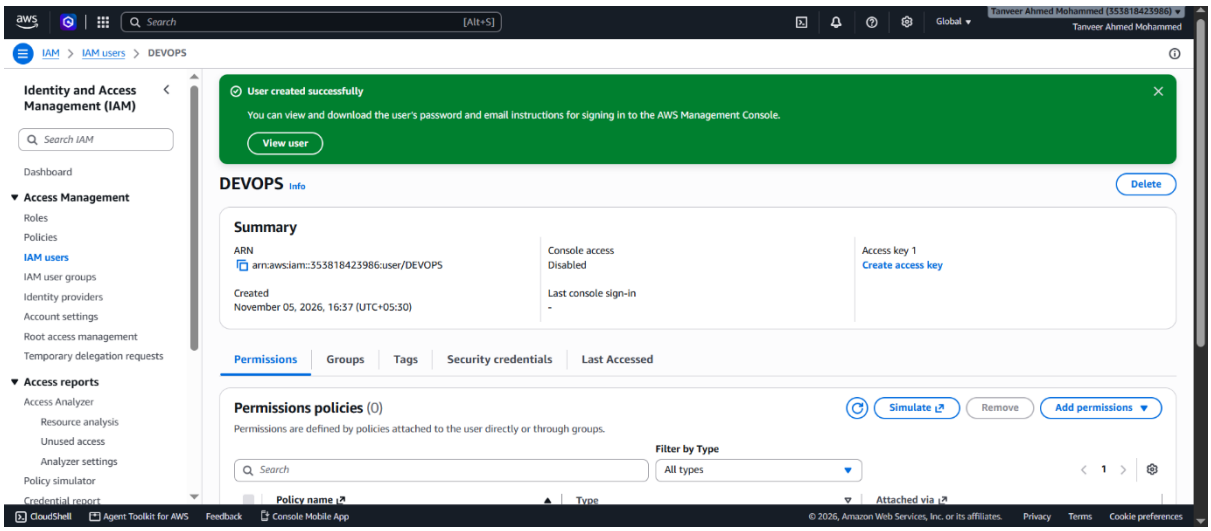
Devops will inherit EC2 read-only permissions from the group.

Step 1: Open IAM Users

1. Login to AWS Management Console.
2. Search for IAM.
3. From the left menu, click Users.
4. Click Create user.

Step 2: Enter username

In the User name field, enter:DEVOPS



Step 3: Add user to the existing group

On the Set permissions screen, you will see three options:

Add user to group

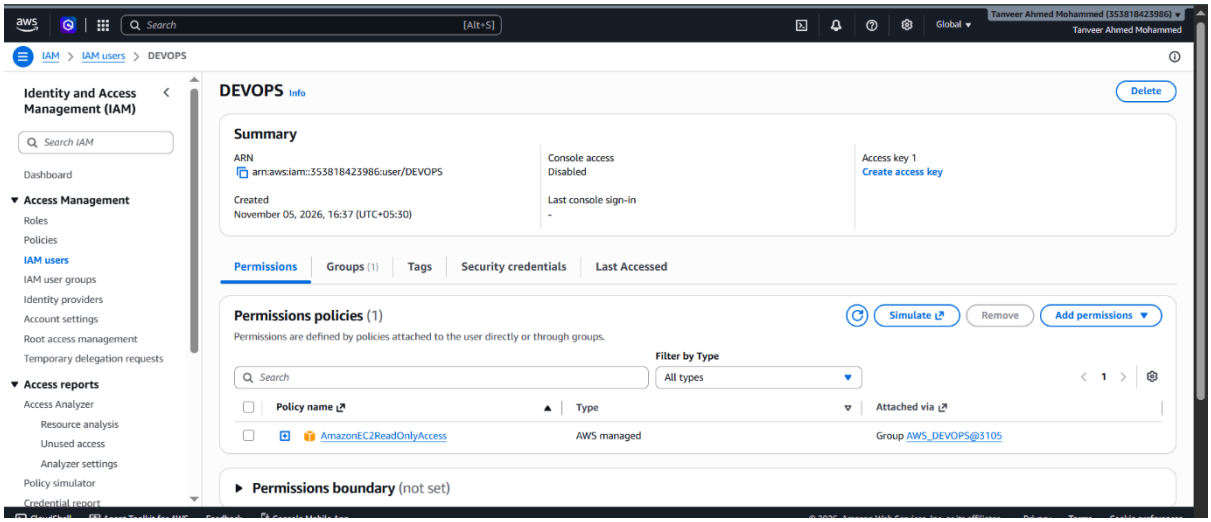
You should see the existing group:

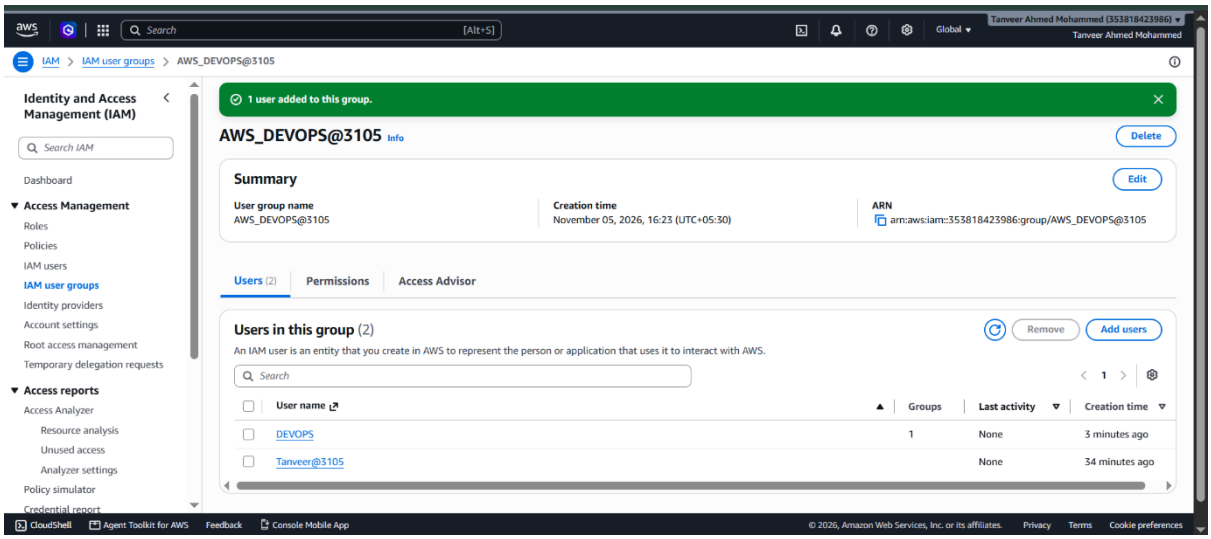
EC2-ReadOnly-GroupSelect its checkbox.

Important: Do not select Attach policies directly. We want the user to inherit permissions from the group.

Step 4: Review and create

1. Click Next.
2. Verify the username is Devops.
3. Verify the group is EC2-ReadOnly-Group.
4. Click Create user.
5.
6. **Step 5: Verify group membership**
7. Navigate to:
8. `IAM → Users → Devops → Groups`





Step 5: Verify group membership

Navigate to:

IAM → Users → Devops → Groups

Expected:

IAM User: Devops

Devops

Group membership

EC2-ReadOnly-Group

Inherited permission

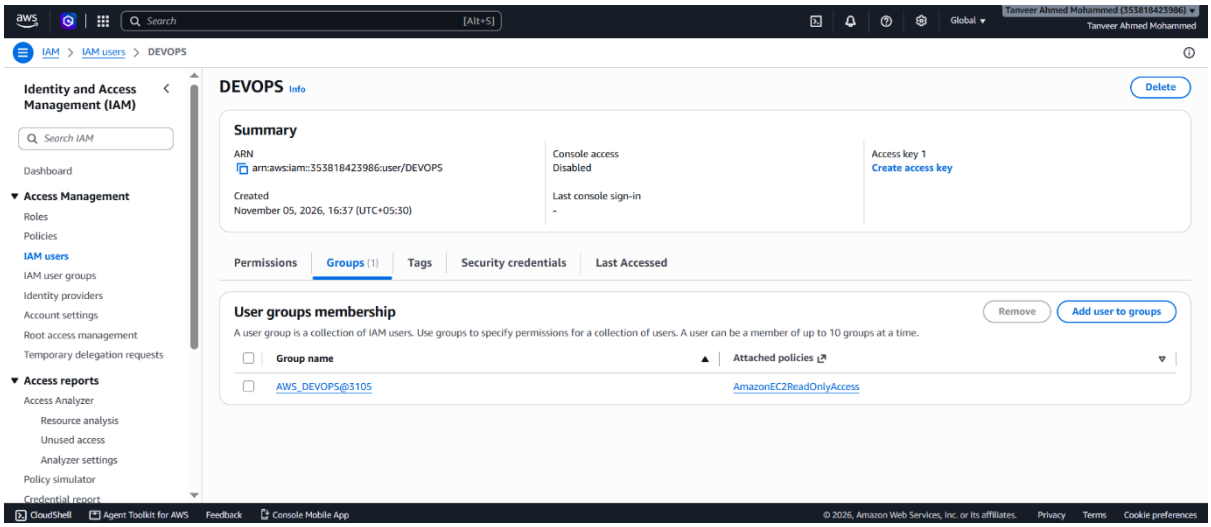
AmazonEC2ReadOnlyAccess

Step 6: Verify permissions

Open:

IAM → Users → Devops → Permissions

Check that AmazonEC2ReadOnlyAccess is inherited through the group.



4)Write a bash script to create an IAM user with VPC full access.

Objective: Create an IAM user automatically using a Bash script and attach the AWS-managed AmazonVPCFullAccess policy.

Here is the complete, step-by-step process to set up your environment, create the IAM user with VPC full access via the script, and verify that everything works correctly. **Step 1: Meet the Prerequisites**

Before running any scripts, ensure you have the following ready:

1. **An AWS Account** with administrative permissions.
2. **AWS CLI installed** on your machine. You can verify this by running:

Bash

aws --version**Step 2: Configure AWS CLI Credentials**

If you haven't already connected your terminal to your AWS account, configure your credentials:

Bash

aws configure

You will be prompted to enter:

- **AWS Access Key ID**: Your admin access key.
- **AWS Secret Access Key**: Your admin secret key.
- **Default region name**: e.g., us-east-1 or your preferred region.
- **Default output format**: e.g., json.

Step 3: Create and Save the Script File

1. Open your terminal and create a new file named

nano create_vpc_user.sh

```
#!/bin/bash

# Exit immediately if a command exits with a non-zero status

set -e

# Configuration variables

USERNAME="vpc-admin-user"

POLICY_ARN="arn:aws:iam::aws:policy/AmazonVPCFullAccess"

echo "Step 1: Creating IAM user '$USERNAME'..."

aws iam create-user --user-name "$USERNAME"

echo "Step 2: Attaching AmazonVPCFullAccess policy..."

aws iam attach-user-policy \

   --user-name "$USERNAME" \

   --policy-arn "$POLICY_ARN"

echo "Success! IAM user '$USERNAME' has been created with VPC Full Access."
```

3. Save and exit the editor (in nano, press Ctrl+O, Enter, then Ctrl+X).

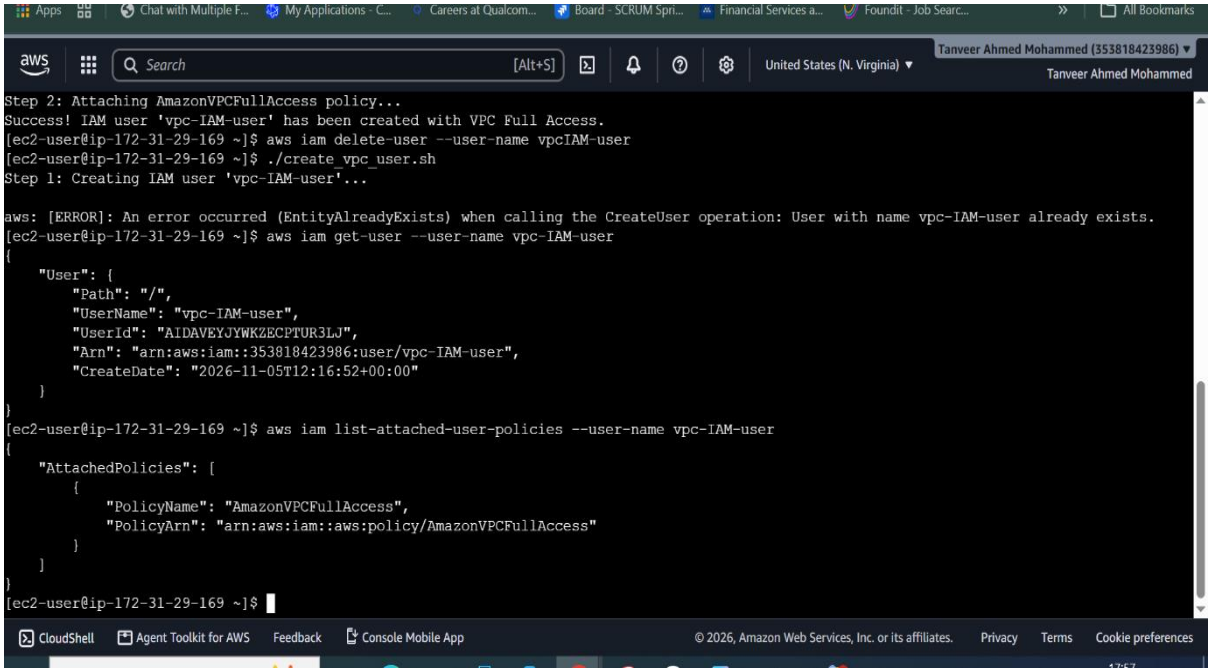
Step 4: Make the Script Executable and Run It

```
chmod +x create_vpc_user.sh

./create_vpc_user.sh
```

Verify the user details:

aws iam get-user --user-name vpc-IAM-user

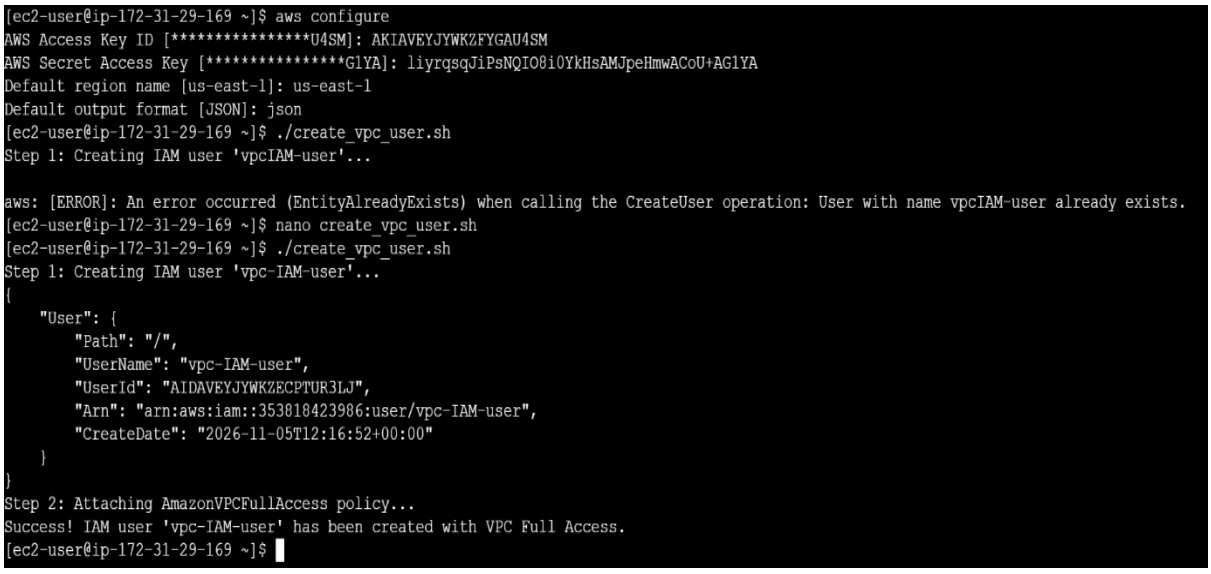


Verification: The command should output a JSON block showing the user's ARN, creation date, and path.

Check attached policies:

aws iam list-attached-user-policies --user-name vpc-IAM-user

Verification: You should see AmazonVPCFullAccess listed in the returned policy array.

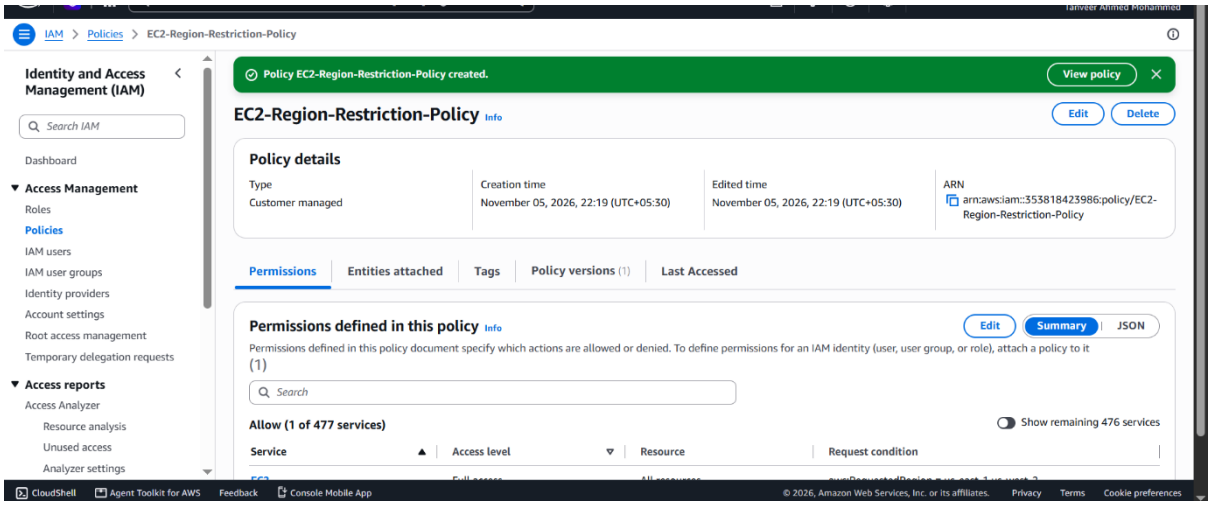


5)Create an IAM policy to allow EC2 access for a specific user in specific regions only.

Phase 1: Create the Region-Restriction Policy

1. Log in to the AWS Management Console with an administrator or IAM-privileged account.
2. In the top search bar, type **IAM** and click on the **IAM** service to open the Identity and Access Management dashboard.

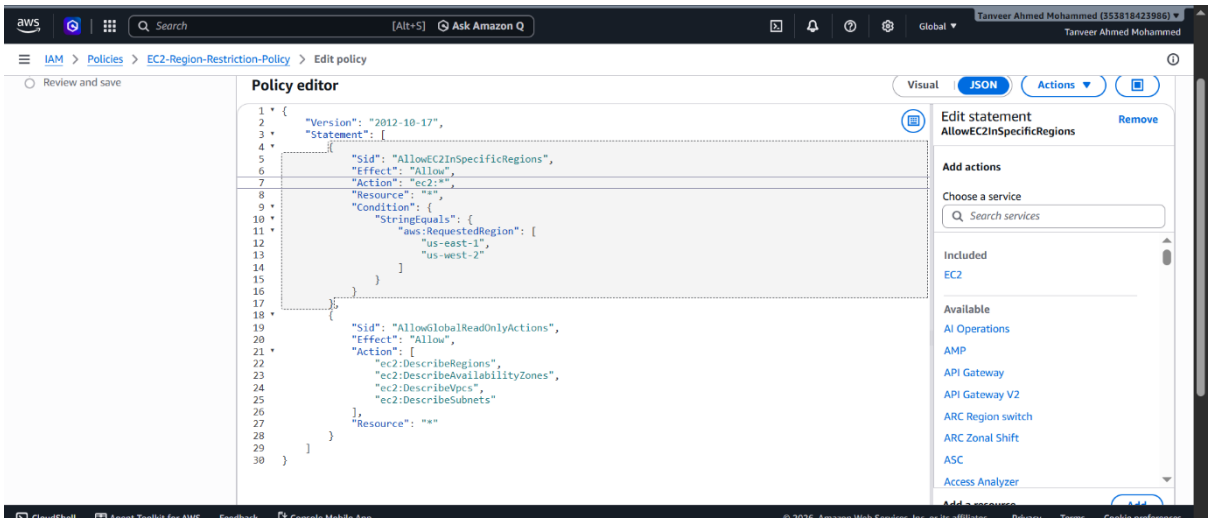
3. In the left-hand navigation menu, click on **Policies** (under *Access management*).
4. Click the orange **Create policy** button in the top-right corner.
5. Select the **JSON** radio button tab, clear any existing text, and paste the exact policy code below:



JSON

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Sid": "AllowEC2InSpecificRegions",
         "Effect": "Allow",
         "Action": "ec2:*",
         "Resource": "*",
         "Condition": {
            "StringEquals": {
               "aws:RequestedRegion": [
                  "us-east-1",
                  "us-west-2"
               ]
            }
         }
      },
```

```
    {
      "Sid": "AllowGlobalReadOnlyActions",
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeRegions",
        "ec2:DescribeAvailabilityZones",
        "ec2:DescribeVpcs",
        "ec2:DescribeSubnets"
      ],
      "Resource": "*"
    }
  ]
}
```



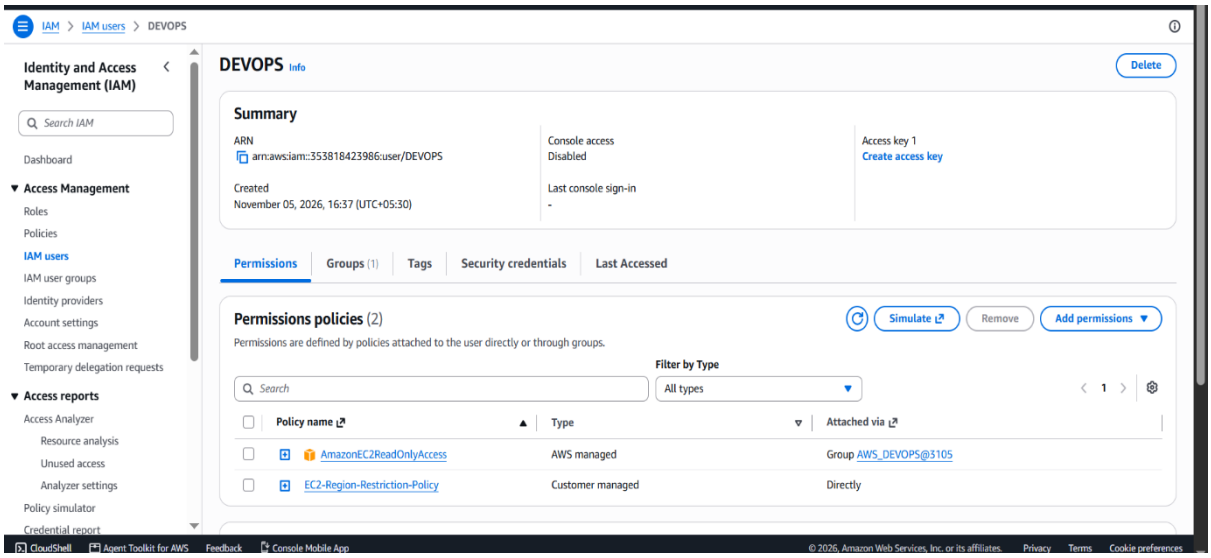
6. Click the **Next** button in the bottom right.
7. Under **Policy details**, enter a clear name like EC2-Region-Restriction-Policy.
8. Review the summary to ensure there are no syntax errors, then click **Create policy**.

- **Verification:** A green notification banner appears at the top stating: *"Successfully created policy EC2-Region-Restriction-Policy."*

Phase 2: Attach the Policy to the Specific User

1. In the left-hand navigation pane of the IAM console, click on **Users**.
2. From the list of users, click on the name of the specific user you want to restrict.
3. On the user summary page, click the **Permissions** tab.
4. Click the **Add permissions** dropdown button and select **Attach policies directly**.

5. In the search box, type EC2-Region-Restriction-Policy.
6. Select the checkbox next to your custom policy.
7. Click **Next**, review the permission summary, and click **Add permissions**.



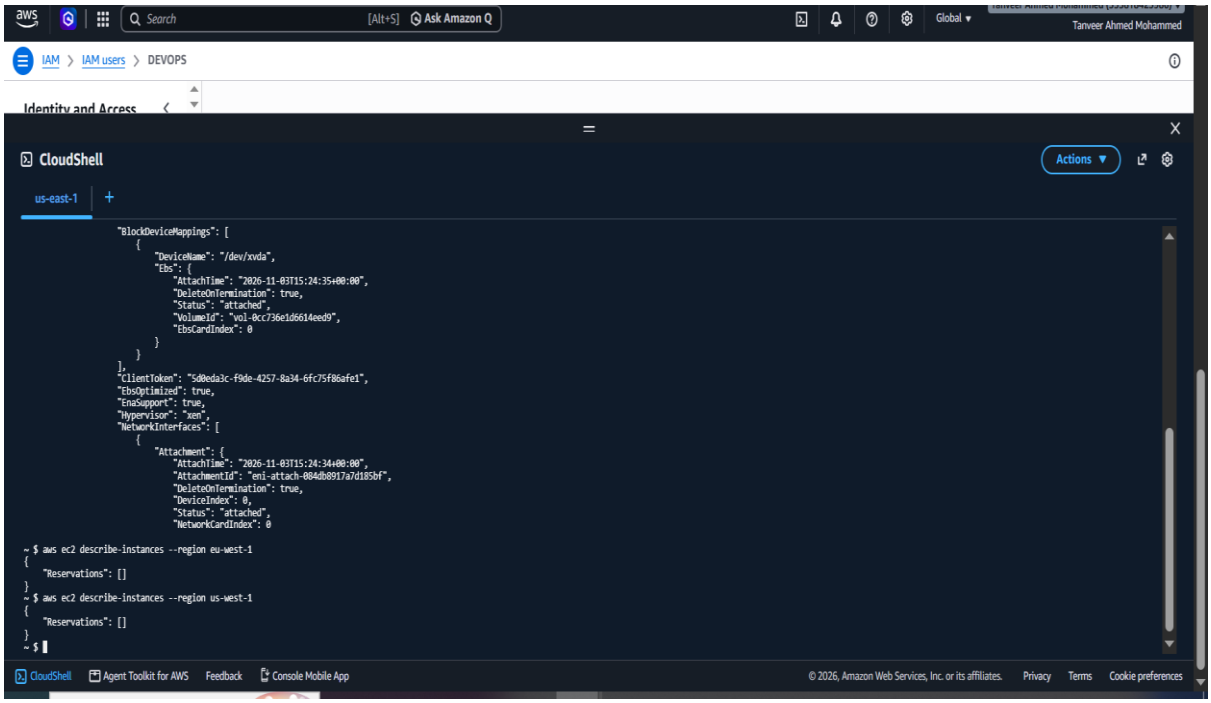
Verification: The policy name EC2-Region-Restriction-Policy now appears under the user's **Permissions policies** list.

Phase 3: Test and Verify the Restrictions

To confirm the policy works correctly, log in using the target user's credentials (or assume their profile via AWS CLI):

1. **Test an Allowed Region (Should Succeed):** Run the following command or check the AWS Console set to us-east-1:

aws ec2 describe-instances --region us-east-1



6)We have two accounts: Account A and Account B. Account A user should access an S3 bucket in Account B.

The Main Purpose is to configure cross-account access in AWS so that a specific user or role in Account A can securely access, read, or write to an Amazon S3 bucket located in Account B without needing to share IAM credentials or create duplicate users.

Why This Is Done in Real-World Environments

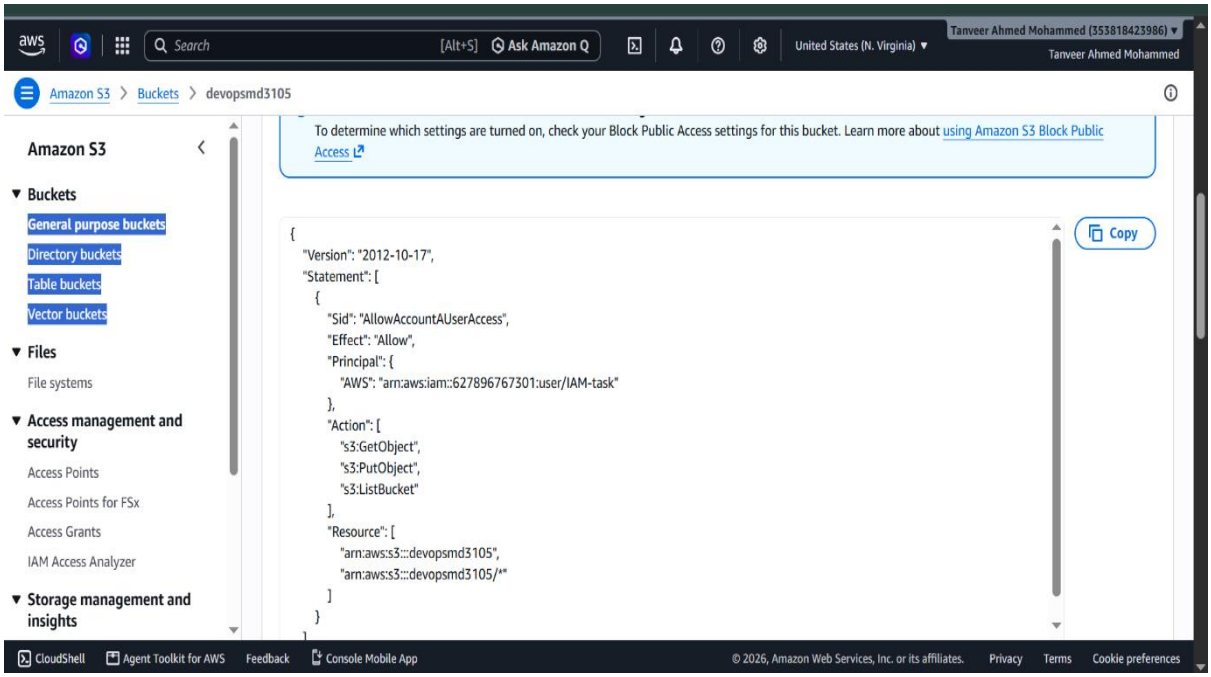
- Centralized Data Management: Companies often store centralized logs, backups, or data lakes in a dedicated security or core account (Account B) while team members work out of separate application or sandbox accounts (Account A).
- Security & Principle of Least Privilege: Instead of making the bucket public or sharing account root credentials, cross-account permissions allow you to establish a secure, explicit trust relationship between two specific AWS accounts.
- Collaboration Without Compromise: Developers or services in one team's account can interact with shared resources in another team's account securely and auditably.

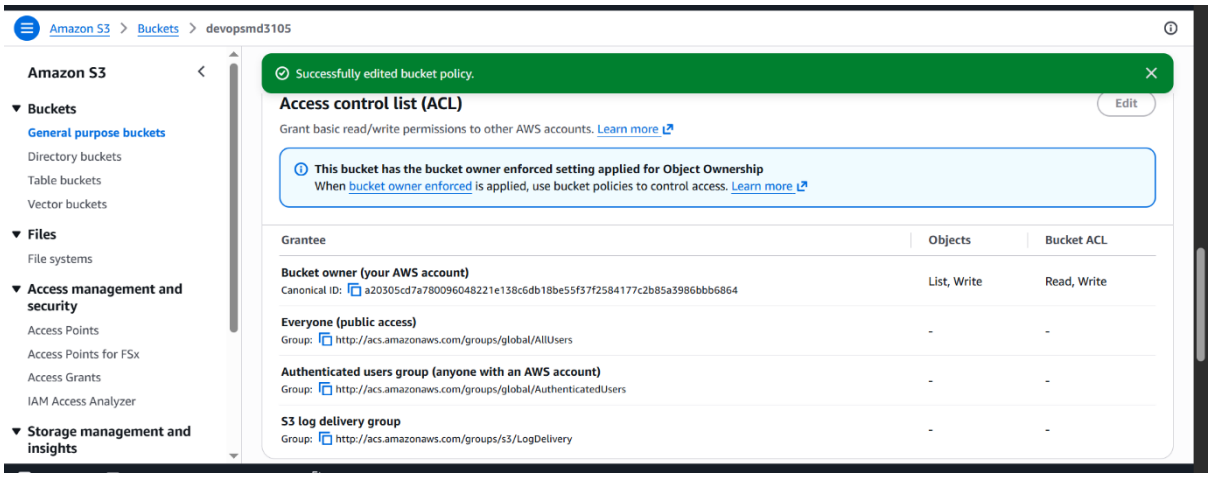
B: -S3 > create a bucket > made a name of bucket > permissions > bucket policy section > edit > paste the below json type policy in s3 bucket (user name: IAM-tash , account ID : 627896767301, Bucket name : devopsmd3105) .

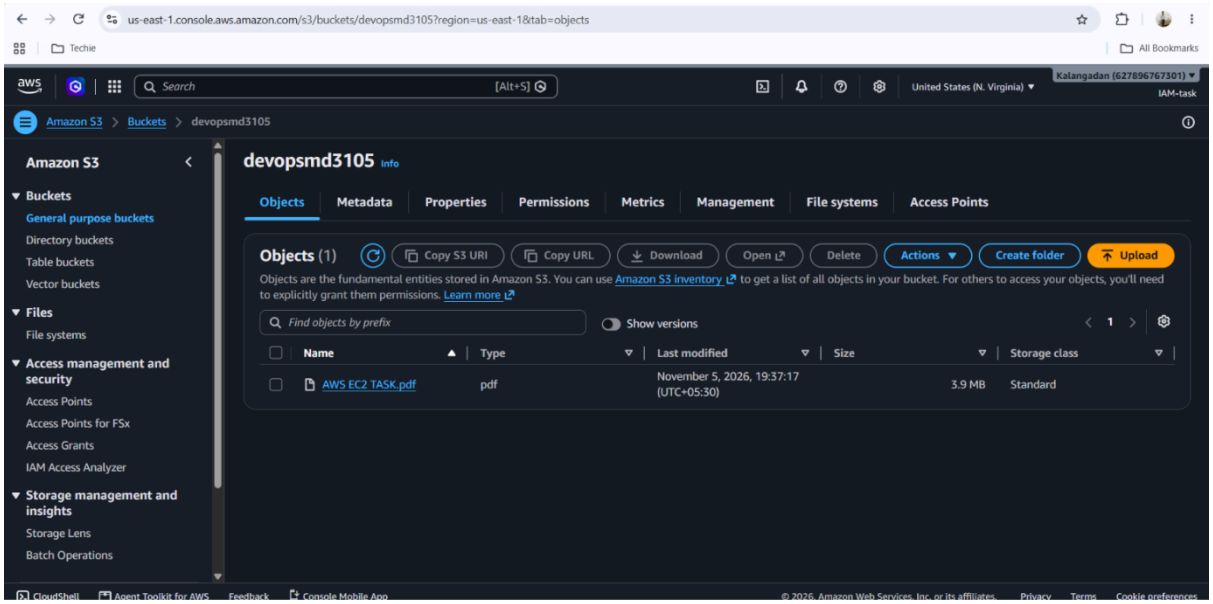
A: login with the given username and credentials then copy the account B bucket URL and now able to see the bucket and details of bucket.

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Sid": "AllowAccountAUserAccess",
         "Effect": "Allow",
         "Principal": {
            "AWS": "arn:aws:iam::627896767301:user/IAM-task"
         },
         "Action": [
            "s3:GetObject",
            "s3:PutObject",
            "s3:ListBucket"
         ],
```

```
        "Resource": [
            "arn:aws:s3:::devopsmd3105",
            "arn:aws:s3:::devopsmd3105/*"
        ]
    }
  ]
}
```







Successfully Bucket seen to Account A.